



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
THAPATHALI CAMPUS**

**A PROJECT PROPOSAL
ON
EXPLAINABLE WIND-AWARE SPATIO-TEMPORAL GRAPH NEURAL NETWORK
FOR AIR QUALITY FORECASTING AND MITIGATION ANALYSIS**

Submitted By:

Aman Singh	(THA079BCT002)
Avinash Joshi	(THA079BCT007)
Kaushik Rayamajhi	(THA079BCT018)
Nirika Lamichhane	(THA079BCT024)

Submitted To:

Department of Electronics and Computer Engineering
Thapathali Campus
Kathmandu, Nepal

June 2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to the Institute of Engineering, Tribhuvan University, for including the major project as an important part of the Bachelor's Degree in Computer Engineering curriculum.

We are also thankful to the Department of Electronics and Computer Engineering, Thapathali Campus, for providing academic support and guidance during the preparation of this project proposal.

Finally, we would like to thank all teachers, friends, and contributors who directly or indirectly supported us during the preparation of this proposal.

Aman Singh	(THA079BCT002)
Avinash Joshi	(THA079BCT007)
Kaushik Rayamajhi	(THA079BCT018)
Nirika Lamichhane	(THA079BCT024)

ABSTRACT

Ambient air pollution is a major environmental and public-health concern in urban areas where pollutant concentration varies across space, time, and season. Kathmandu Valley is affected by road traffic, dust resuspension, construction activities, brick kilns, seasonal forest-fire episodes, valley topography, and changing meteorological conditions, making short-term $PM_{2.5}$ forecasting important for air quality interpretation and public awareness. Traditional station-wise forecasting models often ignore wind-driven pollutant movement between monitoring stations and usually provide numerical outputs without clearly explaining why a forecast is high or low. This proposal presents an explainable wind-aware Spatio-Temporal Graph Neural Network (ST-GNN) framework for short-term $PM_{2.5}$ forecasting, interpretation, and visualization in Kathmandu Valley. Monitoring stations are represented as graph nodes, while dynamic edges are constructed using geographical distance, historical $PM_{2.5}$ correlation, wind speed, and wind-direction alignment. The primary forecasting model is a Graph Attention Network with Gated Recurrent Unit (GAT-GRU), where GAT captures spatial influence between stations and GRU captures temporal patterns from historical observations. The forecasting task is formulated as a regression problem for hourly $PM_{2.5}$ prediction at a 6-hour horizon, while Air Quality Index categories and pollution-peak labels are derived from predicted concentrations to support interpretation and event-level evaluation. The system includes timestamp alignment, missing-value handling, sensor anomaly detection, outlier treatment, normalization, and explainability through attention-based station influence, feature attribution, and temporal sensitivity analysis. The final output will be an interactive dashboard showing station-level $PM_{2.5}$ forecasts, AQI risk classes, historical-versus-predicted trends, spatial pollution maps, virtual-node estimates for unmonitored locations, and feature-importance visualizations. The project is intended as a research prototype and decision-support tool for air quality interpretation and visualization, not as a replacement for official air-quality monitoring or warning systems.

Keywords—*Air quality forecasting, $PM_{2.5}$, wind-aware graph, spatio-temporal graph neural network, GAT-GRU, explainable AI, AQI, Kathmandu Valley.*

TABLE OF CONTENTS

ACKNOWLEDGEMENT	i
ABSTRACT	ii
LIST OF FIGURES	v
LIST OF TABLES	vi
LIST OF ABBREVIATIONS	vi
LIST OF SYMBOLS	vi
1 INTRODUCTION	1
1.1 Background	1
1.2 Motivation	2
1.3 Problem Definition	2
1.4 Objectives	3
1.5 Scope and Applications	3
2 LITERATURE REVIEW	4
2.1 Air Quality Forecasting and the Need for Spatio-Temporal Modeling . . .	4
2.2 Graph Neural Networks for Station-Based Air Quality Forecasting . . .	4
2.3 Spatio-Temporal GNNs and Domain Knowledge in Air Quality Prediction	5
2.4 Wind-Aware and Dynamic Graph Construction	6
2.5 Explainability in Air Quality Forecasting	7
2.6 Data Sources for Pollutant and Meteorological Information	7
2.7 Kathmandu Valley Air Quality Context	8
2.8 Research Gap	8
3 METHODOLOGY	10
3.1 System Block Diagram/Architecture	10
3.2 Description of Working Principle	10
3.3 Flowchart	21
3.4 Instrumentation Tools	22
3.5 Evaluation Methodology	22
4 EXPECTED OUTCOME	24
4.1 Technical Outcomes	24
5 PROJECT SCHEDULE	25

6	PROJECT BUDGET	26
7	FEASIBILITY ANALYSIS	27
7.1	Technical Feasibility	27
7.2	Operational Feasibility	27
7.3	Economic Feasibility	28
7.4	Schedule Feasibility	28
7.5	Risk and Mitigation	28
7.6	Limitations	29
7.7	Final Feasibility Statement	29
	REFERENCES	31

LIST OF FIGURES

Figure 3-1 System Block Diagram	10
Figure 3-2 Spatial modeling using graph attention	18
Figure 3-3 Temporal modeling using GRU	19
Figure 3-4 Proposed air quality forecasting workflow	21
Figure 5-1 Gantt Chart	25

LIST OF TABLES

Table 3-1 Illustrative Sample of the Integrated Dataset	11
Table 3-2 Proposed Technical Stack	22
Table 6-1 Estimated Project Budget	26
Table 7-1 Project Risks and Mitigation Strategies	29

1 INTRODUCTION

1.1 Background

Air pollution is one of the major environmental and public-health concerns in urban areas. Fine particulate matter, commonly known as $PM_{2.5}$, is especially important because it consists of very small airborne particles that can enter deep into the respiratory system and affect human health. $PM_{2.5}$ concentration is influenced by many factors such as vehicular emissions, road dust, construction activities, industrial sources, seasonal variation, temperature, humidity, wind speed, wind direction, and local geography.

Kathmandu Valley is a suitable study area for $PM_{2.5}$ forecasting because it experiences frequent air pollution problems due to dense urban activity, traffic emissions, construction dust, and its bowl-shaped valley structure. The surrounding hills and limited air circulation can contribute to pollutant accumulation under certain meteorological conditions. Therefore, short-term $PM_{2.5}$ forecasting in Kathmandu Valley can support public awareness, environmental monitoring, and air quality interpretation.

Air quality forecasting is a spatio-temporal problem. This means that pollution changes across both location and time. The future $PM_{2.5}$ concentration at one monitoring station may depend not only on its own previous readings, but also on nearby stations, wind movement, and meteorological conditions. Traditional time-series models can learn temporal patterns, but they may not fully capture spatial influence between monitoring stations.

Graph Neural Networks (GNNs) provide a suitable approach for this type of problem because air quality monitoring stations can be represented as a graph. In this graph, each monitoring station is treated as a node, and the relationship between stations is represented as an edge. By combining graph-based spatial learning with recurrent temporal learning, the system can model both station-to-station influence and time-based pollution variation.

In this project, an explainable wind-aware spatio-temporal graph neural network framework is proposed for $PM_{2.5}$ forecasting in Kathmandu Valley. The system will use historical $PM_{2.5}$ observations, meteorological variables, and station relationships to forecast $PM_{2.5}$ concentration after 6 hours. It will also provide explanation outputs to show which stations, environmental features, and previous time patterns influenced the prediction.

1.2 Motivation

Existing air quality forecasting approaches often focus mainly on individual monitoring stations or use fixed spatial relationships between stations. However, pollution does not remain fixed in one location. Wind can transport polluted air from one area to another, and the influence between stations can change depending on wind direction and wind speed. A fixed graph based only on distance may therefore fail to represent the changing nature of pollutant movement.

Kathmandu Valley also has limited monitoring coverage compared to the complexity of its urban environment. Some areas may not have physical air quality sensors, even though people living in those areas still need information about local air quality. This motivates the use of graph-based modeling and virtual nodes, where selected unmonitored map locations can be estimated using nearby real monitoring stations and wind-aware spatial relationships.

Another motivation is explainability. A forecasting system should not only produce a predicted $PM_{2.5}$ value; it should also provide understandable reasons behind the prediction. For example, the system should help explain whether a forecast was influenced by high $PM_{2.5}$ at a nearby station, wind flow from a polluted direction, low wind speed, humidity, or recent pollution trends. Such explanations can make the system more transparent and useful for environmental monitoring, interpretation, and public awareness.

1.3 Problem Definition

Short-term $PM_{2.5}$ forecasting in Kathmandu Valley is challenging because pollutant concentration is affected by both temporal and spatial factors. $PM_{2.5}$ at a particular station may depend on its previous values, meteorological conditions, wind direction, wind speed, and pollutant levels at nearby or upwind stations. Many conventional forecasting methods treat stations independently or use static spatial relationships, which may not properly represent changing pollutant transport caused by wind.

In addition, air quality monitoring stations are limited in number and cannot cover every road, neighborhood, or residential area. This creates difficulty in estimating $PM_{2.5}$ concentration at unmonitored locations. Many forecasting systems also provide only numerical predictions without explaining why the model made a certain forecast.

Therefore, the problem addressed in this project is to design and develop an explainable wind-aware spatio-temporal graph neural network model that can forecast $PM_{2.5}$ concentration after 6 hours for selected Kathmandu Valley monitoring stations and selected virtual locations. The system will model station relationships using distance,

PM_{2.5} correlation, wind direction, and wind speed, and will present interpretable outputs through a dashboard.

1.4 Objectives

The objectives of the project are:

1. To collect and preprocess PM_{2.5} and meteorological data and build a wind-aware dynamic graph using distance, correlation, and wind features.
2. To develop and evaluate baseline and GAT-GRU models for PM_{2.5} forecasting and estimate pollution at unmonitored locations using virtual nodes.
3. To provide model explainability and build a dashboard for visualizing forecasts, Air Quality Index (AQI), spatial patterns, and explanations.

1.5 Scope and Applications

The scope of this project focuses on PM_{2.5} forecasting in Kathmandu Valley using selected monitoring stations from Kathmandu, Lalitpur, and Bhaktapur. The system will use historical PM_{2.5} observations and meteorological variables such as wind speed, wind direction, temperature, humidity, pressure, dew point, and temporal features where available.

The project will represent monitoring stations as graph nodes and model the relationship between stations using distance, PM_{2.5} correlation, and wind-aware features. In addition to physical monitoring stations, selected unmonitored map locations may be represented as virtual nodes to estimate PM_{2.5} levels in areas without direct sensor readings.

The system will be developed as a research prototype for forecasting, explanation, and visualization. The major applications include PM_{2.5} forecasting, station-level air quality interpretation, virtual-node-based estimation for unmonitored locations, pollution risk visualization, explainable environmental monitoring, and academic research support. The dashboard will help users understand predicted PM_{2.5} levels, AQI categories, spatial pollution patterns, important influencing stations, and important environmental features behind each prediction.

2 LITERATURE REVIEW

2.1 Air Quality Forecasting and the Need for Spatio-Temporal Modeling

Air pollution is a serious environmental and public-health issue because pollutant concentration does not remain constant across location or time. Pollutants such as $\text{PM}_{2.5}$, PM_{10} , NO_2 , and O_3 are affected by many factors, including traffic emissions, industrial activities, dust, temperature, humidity, atmospheric pressure, wind speed, and wind direction. Because of this, forecasting air quality is not only a time-series problem but also a spatial problem. The pollution level at one monitoring station may depend on its own past readings, but it may also be influenced by nearby or upwind stations.

Traditional forecasting methods such as statistical models, regression-based models, support vector machines, random forest, Long Short-Term Memory (LSTM), and Gated Recurrent Unit (GRU) have been widely used for environmental prediction. These methods are useful as baseline models because they can learn historical trends and short-term temporal patterns. However, many of them treat each monitoring station independently. In reality, monitoring stations are connected through atmospheric movement and pollutant transport. For example, if pollution is high in one area and wind carries polluted air toward another area, the second area may experience increased pollution after some time. This kind of relationship cannot be fully captured by station-wise models alone.

Recent air quality forecasting studies have therefore moved toward spatio-temporal modeling. In this approach, the model learns both temporal dependency and spatial dependency. Temporal dependency means how pollution changes over hours or days, while spatial dependency means how one location may influence another. This idea is important for the proposed project because the system aims to forecast pollutant levels by combining historical pollution data, meteorological data, and relationships between monitoring stations.

2.2 Graph Neural Networks for Station-Based Air Quality Forecasting

GNNs provide a suitable framework for modeling air quality monitoring networks. In a graph-based representation, each air quality monitoring station can be considered as a node, and the relationship between two stations can be represented as an edge. This allows the model to learn how stations interact with each other rather than treating every station as completely independent.

The Graph Convolutional Network proposed by Kipf and Welling introduced an efficient method for learning from graph-structured data by aggregating information from neighboring nodes [1]. In the context of air quality forecasting, this means that a

monitoring station can learn from its connected neighboring stations. This is useful because air pollution at one station may be affected by pollution conditions in surrounding areas.

However, all neighboring stations do not contribute equally. Some stations may have stronger influence because they are closer, more polluted, or located in the direction from which wind is blowing. Graph Attention Network, proposed by Velickovic et al., addresses this issue by allowing the model to assign different attention weights to different neighboring nodes [2]. This makes graph attention useful for air quality forecasting because the model can learn which neighboring station is more important for a particular prediction.

These graph-based approaches provide the foundation for the proposed project. Graph convolution can be used as a baseline spatial model, while graph attention can be used to learn unequal station influence. When combined with temporal models such as GRU or LSTM, these methods can support spatio-temporal air quality forecasting.

2.3 Spatio-Temporal GNNs and Domain Knowledge in Air Quality Prediction

Several recent studies have shown that air quality forecasting improves when graph learning is combined with domain knowledge and temporal modeling. Wang et al. proposed PM_{2.5}-GNN, a domain-knowledge enhanced graph neural network for PM_{2.5} forecasting [3]. Their work highlights that PM_{2.5} concentration is influenced by complex environmental factors and that graph-based models can capture relationships between locations more effectively than simple time-series models. This supports the idea that air quality forecasting should include not only historical pollutant values but also spatial and meteorological relationships.

Pandey et al. proposed a spatial-diffusion guided encoder-decoder architecture for PM_{2.5} forecasting [4]. Their model combines temporal learning with graph-based spatial diffusion, showing that PM_{2.5} forecasting benefits from modeling how pollution spreads across monitoring locations. This is closely related to the proposed project because the planned system also combines graph-based spatial learning with recurrent temporal learning.

Panja et al. proposed E-STGCN, an extreme spatio-temporal graph convolutional network for air quality forecasting [5]. Their work focuses on high-pollution or extreme pollution events, which are important in public-health decision-making. Although the proposed project does not directly focus on extreme value theory, this study is useful because it shows that spatio-temporal graph models can be extended for risk-aware air quality forecasting.

These studies confirm that spatio-temporal GNNs are a strong direction for air quality prediction. However, a major limitation in many graph-based forecasting systems is that the graph structure is often fixed. A fixed graph may be created using distance or historical correlation, but real pollutant movement changes with wind and weather. This creates the need for dynamic and wind-aware graph construction.

2.4 Wind-Aware and Dynamic Graph Construction

The main technical idea of the proposed project is that the relationship between two air quality stations should not always remain fixed. In real atmospheric conditions, wind direction and wind speed can change throughout the day. If wind flows from one station toward another, the first station may have stronger influence on the second station. If the wind changes direction, the influence may also change. Therefore, a graph used for air quality forecasting should ideally be dynamic and wind-aware.

HighAir, proposed by Chen et al., is an important reference in this direction because it uses a hierarchical graph neural network for air quality forecasting and dynamically adjusts graph edge weights based on wind direction [6]. This directly supports the proposed project's idea that wind direction should influence station-to-station relationships.

Xu et al. proposed a Dynamic Graph Neural Network with Adaptive Edge Attributes for air quality prediction [7]. Their work argues that predefined graph structures may not always represent real relationships between monitoring sites. Instead, the model should learn or adjust edge attributes dynamically. This is important for the proposed system because the planned graph will not depend only on fixed distance. It will combine distance, historical pollutant correlation, wind direction, and wind speed to represent changing station influence.

The idea of dynamic graph construction is also connected to the physical behavior of pollutants. Air pollutants spread through diffusion and can be transported by wind through advection. AirPhyNet, proposed by Hettige et al., uses physics-guided neural networks for air quality prediction and incorporates physical ideas such as diffusion and advection [8]. This provides scientific support for the proposed wind-aware graph approach. Even though the proposed project may not fully implement a physics-guided differential equation model, the use of wind-based edge weighting follows the same physical intuition that pollutants move through the atmosphere rather than changing randomly.

2.5 Explainability in Air Quality Forecasting

Accuracy alone is not enough for an environmental decision-support system. If a model predicts that $PM_{2.5}$ will become high in a certain area, users should also understand why the model made that prediction. This is especially important in air quality forecasting because the prediction may influence public awareness, planning, and mitigation discussion.

GNNExplainer, proposed by Ying et al., provides a method for explaining predictions made by graph neural networks [9]. It identifies important parts of the graph and important node features that contribute to a prediction. In the proposed project, this idea can help identify which neighboring stations had the strongest influence on a forecasted pollution value.

However, station influence is only one part of the explanation. The system should also explain the role of meteorological and pollutant features such as wind speed, wind direction, humidity, temperature, and previous $PM_{2.5}$ values. SHAP, proposed by Lundberg and Lee, provides a general approach for explaining machine-learning predictions through feature contribution values [10]. This can be used to show which input features contributed most to a forecast.

By combining graph-based explanation and feature-level importance analysis, the proposed system can produce more meaningful outputs. Instead of only showing a forecasted $PM_{2.5}$ value, the system can explain that the prediction was influenced by rising $PM_{2.5}$ in previous hours, wind flow from a polluted neighboring station, low wind speed, or high humidity. This makes the model more understandable and useful for practical decision support.

2.6 Data Sources for Pollutant and Meteorological Information

The proposed system requires both air quality data and weather data. OpenAQ provides access to air quality measurements from monitoring stations and can be used to obtain pollutant data such as $PM_{2.5}$, PM_{10} , NO_2 , and O_3 depending on station availability [11]. This is useful for building a reproducible data pipeline using publicly available air quality records.

Meteorological data is equally important because weather strongly affects pollutant concentration and movement. Open-Meteo provides historical weather data such as temperature, humidity, pressure, wind speed, wind direction, and wind gusts [12]. In the proposed project, wind speed and wind direction are especially important because they will be used to construct the wind-aware dynamic graph. Other weather variables can also help the model understand pollution accumulation and dispersion patterns.

2.7 Kathmandu Valley Air Quality Context

Since the project may later focus on Kathmandu Valley, it is important to understand the local air pollution context. Kathmandu Valley is a suitable case study because it has serious air pollution problems, high population density, rapid urbanization, and a valley-shaped geography that can affect pollutant accumulation. Becker et al. studied particulate matter variability in Kathmandu using in-situ measurements, remote sensing, and reanalysis data [13]. Their work is useful because it shows that Kathmandu's particulate matter pollution has strong temporal variability and is affected by seasonal and meteorological conditions.

Panday and Prinn studied the diurnal cycle of air pollution in Kathmandu Valley [14]. Their work is important because it shows that air pollution in Kathmandu changes throughout the day and is influenced by local meteorology and valley circulation. This supports the need for temporal modeling and wind-aware analysis in Kathmandu-focused air quality prediction.

Clean Energy Nepal also highlights that Kathmandu Valley is vulnerable to air pollution because of its bowl-shaped topography, limited wind flow, long dry season, rapid urbanization, and major emission sources such as transport, road dust, construction, brick kilns, household energy use, and waste burning [15]. This local context supports the motivation for developing a forecasting and visualization system for Kathmandu Valley.

ICIMOD reports that Nepal approved the Kathmandu Valley Air Quality Management Action Plan in 2020 and emphasizes the need for better air quality management and decision-support mechanisms [16]. This supports the practical relevance of the proposed dashboard and scenario-based analysis module. The proposed system will not replace official government warning systems, but it can serve as an academic research prototype for forecasting, explanation, visualization, and decision-support.

2.8 Research Gap

The reviewed literature shows that air quality forecasting has advanced significantly through deep learning and graph neural networks. Existing studies have shown that GNNs can model relationships between monitoring locations, domain knowledge can improve $PM_{2.5}$ prediction, and spatio-temporal models can capture both time-based and location-based pollution patterns. Studies such as $PM_{2.5}$ -GNN, HighAir, dynamic graph neural networks with adaptive edge attributes, and AirPhyNet provide strong support for graph-based, wind-aware, and physically meaningful air quality forecasting.

However, several gaps still remain. First, many forecasting systems use fixed graph structures, even though pollutant movement changes with wind direction and wind

speed. Second, many models focus mainly on prediction accuracy but provide limited explanation of why a certain forecast was made. Third, existing systems often do not combine forecasting, explainability, risk visualization, and scenario-based decision support into one integrated dashboard. Finally, although Kathmandu Valley has been studied in terms of particulate matter variability, diurnal pollution cycles, and policy needs, there is limited work applying explainable wind-aware spatio-temporal graph neural networks to Kathmandu Valley air quality forecasting.

The proposed project addresses these gaps by developing an explainable wind-aware spatio-temporal graph neural network framework for air quality forecasting. The system will represent monitoring stations as graph nodes and construct dynamic edges using distance, pollutant correlation, wind speed, and wind direction. It will combine graph-based spatial learning with temporal models such as GRU or LSTM to forecast pollutant concentration. It will also include an explainability module to identify influential stations and important weather features. Finally, the results will be shown through an interactive dashboard for air quality visualization, risk analysis, and what-if scenario-based decision support.

3 METHODOLOGY

3.1 System Block Diagram/Architecture

The proposed system is organized into data collection, preprocessing, wind-aware dynamic graph construction, spatio-temporal forecasting, explainability, storage, backend service, and dashboard layers. Figure 3-1 shows the complete system architecture and how the components interact.

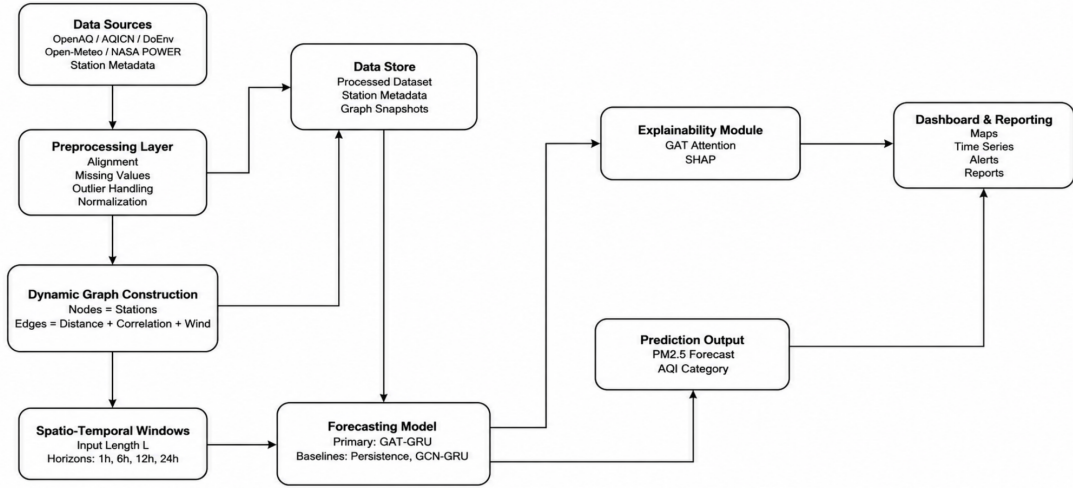


Figure 3-1 System Block Diagram

3.2 Description of Working Principle

The proposed system works as a wind-aware air quality forecasting and decision-support system for short-term PM_{2.5} prediction. It combines air quality data, meteorological data, dynamic graph construction, spatio-temporal deep learning, explainability, and dashboard-based visualization. The overall working principle of the system is described in the following subsections.

3.2.1 Study Area, Stations and Data Access Status

The initial study area for this project will be Kathmandu Valley, Nepal, covering major urban and semi-urban areas of Kathmandu, Lalitpur, and Bhaktapur. Kathmandu Valley is selected as the primary study area because it experiences high air pollution due to traffic emissions, construction activities, industrial sources, dust, seasonal variation, and its bowl-shaped geography, which can trap pollutants.

In the first stage of the project, the study will focus on selected air quality monitoring stations within Kathmandu Valley. These stations will be used to collect historical pollutant data such as PM_{2.5} and PM₁₀, along with meteorological data such as wind

speed, wind direction, temperature, humidity, and pressure where available. The initially considered stations include Ratnapark, Shankapark, TU Kirtipur, Khumaltar, Bhaisipati, Pulchowk, Bhaktapur/Birendra School, and the US Embassy monitoring station. These stations are selected because they provide coverage across different parts of the valley, including dense urban areas, residential zones, and surrounding valley regions.

For model development, each monitoring station will be represented as a node in a graph. The relationship between stations will be defined using factors such as geographical distance, wind direction, wind speed, and pollutant similarity.

3.2.2 Dataset Description

The currently downloaded OpenAQ station datasets include $PM_{2.5}$, PM_1 , relative humidity, and temperature. $PM_{2.5}$ concentration will serve as the target variable, whereas the remaining available variables will be used as input features. Wind speed, wind direction, atmospheric pressure, and dew point obtained from the Open-Meteo API will be considered as additional meteorological features. Wind speed and wind direction are particularly important for modeling the transport of airborne pollutants between monitoring stations. The spatial graph will therefore account for both geographical proximity and time-varying meteorological conditions.

Table 3-1 presents the proposed structure of the integrated dataset. The values are illustrative examples included only to demonstrate the organization of the model-ready records.

Table 3-1 Illustrative Sample of the Integrated Dataset

Timestamp	Station	$PM_{2.5}$	PM_1	Wind Speed	Other Features
2026-01-01 01:00	Ratnapark	72.4	41.2	1.8	RH, temperature, pressure, dew point, wind direction
2026-01-01 01:00	Khumaltar	58.7	33.5	2.1	RH, temperature, pressure, dew point, wind direction
2026-01-01 01:00	Bhaktapur	64.9	37.8	1.5	RH, temperature, pressure, dew point, wind direction

In Table 3-1, $PM_{2.5}$ and PM_1 concentrations are expressed in $\mu g/m^3$, relative humidity is expressed as a percentage, temperature and dew point are expressed in degrees Celsius,

atmospheric pressure is expressed in hPa, wind speed is expressed in m/s, and wind direction is expressed in degrees. The displayed values are illustrative rather than actual observations.

3.2.3 Input Variables, Resolution and Prediction Target

The proposed model will use hourly air quality and meteorological observations from multiple monitoring stations. Let N represent the number of monitoring stations and F represent the number of input features. At each time step t , the observations are represented as a node feature matrix:

$$X_t \in \mathbb{R}^{N \times F}$$

where each row represents a monitoring station and each column represents a pollutant, meteorological, or temporal feature.

To capture temporal patterns, the model will use a sliding window of past observations. The default input window will be 24 hours, with a 48-hour window also tested for comparison. The primary forecasting horizon is 6 hours. Additional horizons of 1, 12, and 24 hours will be tested for comparison.

The main input variables will include PM_{2.5}, PM₁, temperature, humidity, wind speed, wind direction, and temporal features such as hour of day, day of week, month, and season. Wind direction will be converted into sine and cosine components because it is a circular variable. Missingness indicators may also be added to show whether a value was observed or imputed.

The primary prediction target is future PM_{2.5} concentration at each monitoring station. For station i and forecasting horizon h , the target value is represented as:

$$y_i^{(t+h)}$$

and the model prediction is represented as:

$$\hat{y}_i^{(t+h)}$$

Therefore, the task is formulated as a regression problem, where the model learns to minimize the difference between actual and predicted PM_{2.5} values across all stations and time steps. Although the model predicts numerical PM_{2.5} values, air quality

categories such as good, moderate, unhealthy, and hazardous will be generated later in the post-processing stage using standard AQI threshold values.

3.2.4 Data Preprocessing

The collected air quality and meteorological data undergo a preprocessing stage before being used for graph construction and model training. Since the data are obtained from multiple sources, including OpenAQ and Open-Meteo, preprocessing is necessary to ensure consistency, reliability, and compatibility across all monitoring stations and environmental variables.

Air quality observations and meteorological records are first aligned according to their timestamps. Since the proposed system operates on hourly observations, all records are resampled and synchronized to a common hourly interval. For each monitoring station, $PM_{2.5}$, PM_{10} , temperature, humidity, wind speed, and wind direction measurements corresponding to the same timestamp are merged into a unified observation record. This temporal alignment ensures that all input variables represent the same environmental conditions at a given time step and prevents inconsistencies during model training.

The collected datasets are inspected for missing observations. Any missing values identified within the pollutant or meteorological variables are handled using appropriate interpolation techniques. Short gaps in continuous measurements are filled through linear interpolation, while larger missing segments may be removed if sufficient neighboring information is unavailable. This process helps preserve temporal continuity while reducing information loss caused by incomplete observations.

Environmental monitoring data may contain abnormal values caused by sensor malfunctions, communication errors, or measurement noise. Therefore, the dataset is examined for unrealistic or extreme observations. Basic statistical analysis and visualization techniques are used to identify potential outliers. Records containing physically impossible values, such as negative pollutant concentrations or invalid meteorological measurements, are removed or corrected according to data quality guidelines using either capping or log transformation.

The collected variables have different numerical ranges and units. To ensure stable neural network training, all numerical features are normalized before being supplied to the forecasting model. Min-Max normalization is applied to transform each feature into a common numerical range between 0 and 1. The normalized value of each feature is computed as:

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

where X represents the original feature value, X_{min} is the minimum observed value, and X_{max} is the maximum observed value within the training dataset.

After preprocessing and normalization, the sequential observations are converted into fixed-length temporal windows for forecasting. A sliding-window approach is used to construct input sequences from historical observations. For example, if a 24-hour look-back period is selected, the model receives pollutant and meteorological observations from the previous 24 hours and predicts the $PM_{2.5}$ and PM_1 concentration for the next forecasting horizon.

The final output of the preprocessing stage is a clean, synchronized, normalized, and temporally structured dataset containing station-wise pollutant and meteorological observations. These processed records are subsequently used for dynamic graph construction and spatio-temporal forecasting.

3.2.5 Wind-Aware Dynamic Graph Construction

The proposed system models air quality monitoring stations as a dynamic graph in order to capture spatial and meteorological dependencies among different locations within Kathmandu Valley. Unlike conventional static graphs that use only fixed geographical distance or historical correlation, the proposed approach constructs a wind-aware dynamic graph that adapts edge relationships based on time-varying meteorological conditions.

Let the air quality monitoring network be defined as a graph:

$$G_t = (V, E_t, A_t)$$

where V represents the set of monitoring stations, E_t represents the set of edges at time t , and A_t represents the adjacency matrix at time t . Each node $v_i \in V$ corresponds to a monitoring station and is associated with a feature vector:

$$X_i^t = [PM_{2.5}, PM_1, Temperature, Humidity, WindSpeed, WindDirection]$$

An edge between two stations i and j is defined based on geographical distance, historical pollutant correlation, wind speed, and wind direction alignment. Thus, the edge weight is defined as a composite function:

$$A_{ij}^t = f_d(i, j) \cdot f_c(i, j) \cdot f_w(i, j, t)$$

Spatial proximity between stations is modeled using normalized inverse distance:

$$f_d(i, j) = \exp\left(-\frac{d_{ij}}{\sigma_d}\right)$$

where d_{ij} is the geographical distance between stations i and j , and σ_d is a scaling parameter controlling spatial decay.

To capture long-term similarity in pollution patterns, historical correlation is computed as:

$$f_c(i, j) = \text{corr}(PM2.5_i, PM2.5_j)$$

The key contribution of the proposed system is the incorporation of wind-driven pollutant transport. Wind influence is modeled as:

$$f_w(i, j, t) = \alpha \cdot \cos(\theta_{ij} - \theta_t) \cdot WS_t$$

where θ_{ij} is the direction from station i to station j , θ_t is the wind direction at time t , WS_t is the wind speed at time t , and α is a scaling factor. This formulation increases influence when wind flows from one station toward another, decreases influence when wind flows in the opposite direction, and amplifies spatial propagation effects during stronger wind conditions.

The final adjacency matrix is time-dependent and is updated at every time step to reflect changing meteorological conditions:

$$A_t = \begin{bmatrix} A_{11}^t & A_{12}^t & \cdots & A_{1n}^t \\ A_{21}^t & A_{22}^t & \cdots & A_{2n}^t \\ \vdots & \vdots & \ddots & \vdots \\ A_{n1}^t & A_{n2}^t & \cdots & A_{nn}^t \end{bmatrix}$$

The output of this module is the dynamic adjacency matrix A_t , the node feature matrix X_t , and the edge-weighted graph representation G_t . These outputs are subsequently used as input to the graph attention layer for spatial feature learning.

3.2.6 Virtual Node Construction for Unmonitored Locations

A key novelty of this project is the use of virtual nodes to estimate air pollution at locations where physical air quality monitoring stations are not available. In real-world settings, monitoring stations are limited and cannot represent every road, neighborhood, or residential area. To address this limitation, the proposed system will allow selected map locations to be represented as virtual nodes within the graph.

A virtual node represents an unmonitored geographic location selected from the map. Unlike physical stations, this node does not have direct sensor readings. Therefore, its initial feature values will be estimated using information from nearby monitoring stations. The connection between the virtual node and real stations will be determined using geographical distance, wind direction, wind speed, and pollutant similarity. Stations that are closer or located along the wind flow direction will be assigned stronger influence.

Formally, if there are N real monitoring stations and V virtual nodes, the graph is expanded as:

$$N' = N + V$$

where N' is the total number of real and virtual nodes in the graph. The model then performs prediction over both physical monitoring stations and virtual nodes.

The virtual node mechanism enables the system to estimate $PM_{2.5}$ concentration at unmonitored locations and generate a more continuous air pollution map. This is useful for identifying pollution patterns in areas without sensors and for improving the practical usability of the forecasting system.

To validate this approach, a leave-one-station-out strategy may be used. In this method, one real station is temporarily treated as a virtual node, and its actual $PM_{2.5}$ values are hidden from the model during prediction. The predicted values are then compared with the actual station readings. This allows the performance of virtual-node-based prediction to be evaluated even when real ground truth is not available for completely unmonitored locations.

3.2.7 Spatio-Temporal Forecasting Model

The proposed system employs a hybrid Graph Attention Network and Gated Recurrent Unit architecture to model both spatial and temporal dependencies in air quality data. This combination enables the model to capture pollutant interactions across monitoring stations as well as temporal variations over time.

In addition to forecasting $\text{PM}_{2.5}$ at physical monitoring stations, the model will also support prediction at virtual nodes. These virtual nodes represent unmonitored map locations and are connected to the existing station graph using wind-aware spatial relationships. This allows the GAT-GRU model to estimate pollution levels beyond the available sensor network.

At each time step t , the model receives a node feature matrix:

$$X_t \in \mathbb{R}^{N \times F}$$

and a dynamic adjacency matrix:

$$A_t \in \mathbb{R}^{N \times N}$$

where N represents the number of monitoring stations and F represents the number of input features per station. The adjacency matrix is generated from the wind-aware graph using factors such as geographical distance, wind direction, wind speed, and pollutant similarity.

The Graph Attention Network captures spatial influence between monitoring stations. For each station i , the GAT layer aggregates information from neighboring stations using attention weights:

$$h_i^t = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^t W x_j^t \right)$$

where x_j^t is the feature vector of neighboring station j , W is a learnable weight matrix, α_{ij}^t is the attention coefficient, $\mathcal{N}(i)$ represents the neighboring stations of station i , and σ is a non-linear activation function. The attention coefficient α_{ij}^t shows how much influence station j has on station i at time t . This also supports model explainability because the learned attention weights can help identify which nearby stations contribute most to a prediction.

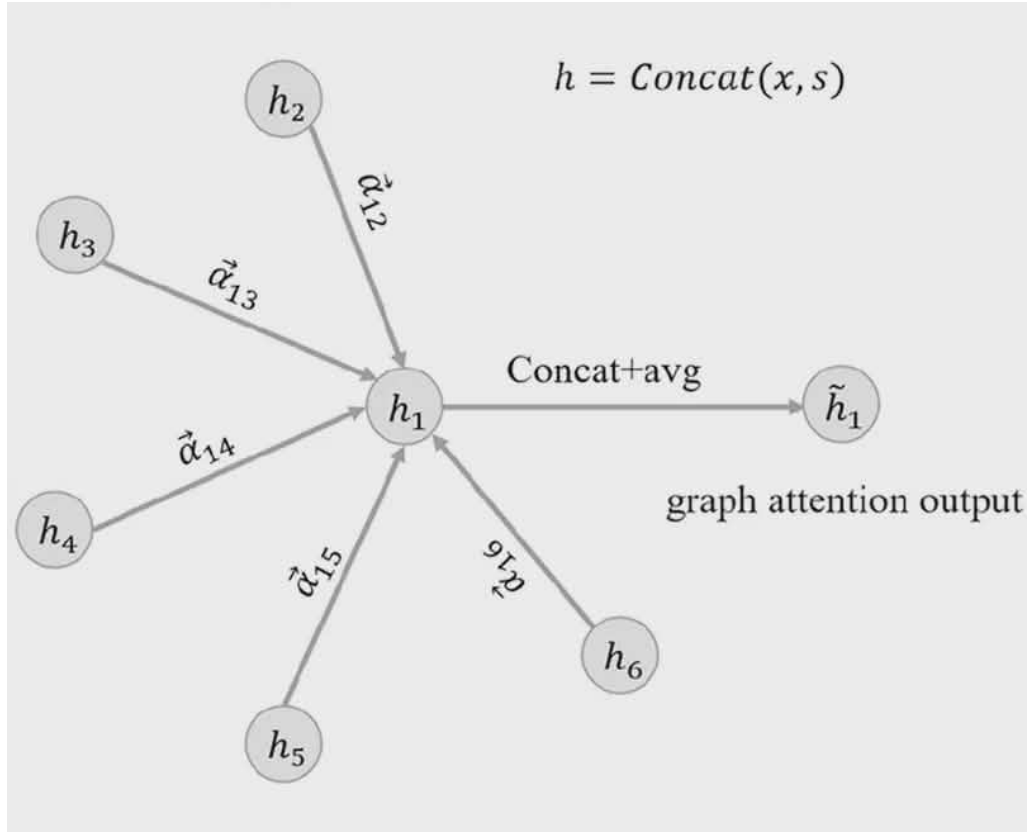


Figure 3-2 Spatial modeling using graph attention

The spatial output of the GAT layer is represented as:

$$H_t = GAT(X_t, A_t)$$

To capture temporal dependencies, the spatial embeddings from multiple previous time steps are passed into a GRU network. For an input window of length L , the GRU processes:

$$H_{t-L+1:t}$$

The hidden state is updated as:

$$z_t = GRU(H_t, z_{t-1})$$

where H_t is the spatial embedding at time t , and z_t is the hidden state that captures temporal evolution. This allows the model to learn short-term pollution changes, delayed meteorological effects, and temporal persistence in $PM_{2.5}$ concentration.

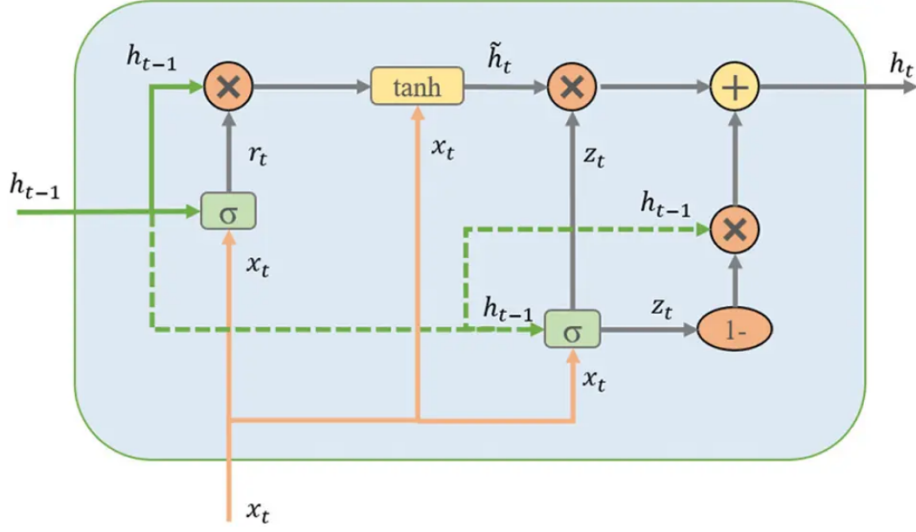


Figure 3-3 Temporal modeling using GRU

The final hidden representation from the GRU is passed through a fully connected regression layer to predict future $\text{PM}_{2.5}$ concentration at each station:

$$\hat{y}_{t+h} = W_o z_t + b$$

where $\hat{y}_{t+h}$ is the predicted $\text{PM}_{2.5}$ concentration after horizon h , and $h \in \{1, 6, 12, 24\}$. The one-hour horizon represents the single-step forecasting case, while 6, 12, and 24-hour horizons are used for multi-horizon forecasting. The overall architecture can be summarized as input features, wind-aware graph construction, GAT layer, GRU layer, and regression output.

3.2.8 Explainability Module

The proposed system includes an explainability module to help interpret the predictions generated by the wind-aware spatio-temporal forecasting model. Since the model uses GAT and GRU layers, the explanation focuses on spatial influence, feature contribution, and temporal importance.

Spatial explainability is obtained from the attention weights learned by the GAT layer. For a target station or virtual node, the attention coefficient α_{ij} shows how much neighboring station j contributes to the prediction at node i . These attention values help identify the most influential stations, possible pollutant movement direction, and the effect of wind-aware graph connections. This is especially useful when estimating $\text{PM}_{2.5}$ at virtual nodes, as it can show which real monitoring stations influenced the predicted value.

Feature-level explainability will be analyzed using SHAP or similar feature importance methods. This will help identify how much each input variable, such as past $\text{PM}_{2.5}$, PM_{10} , wind speed, wind direction, temperature, humidity, and temporal features, contributes to the final prediction. This allows the system to explain why a predicted pollution value is high or low.

Temporal interpretability will be studied by analyzing how different historical time windows affect the forecast. Since the GRU learns from past observations, this analysis helps understand whether the prediction is mainly influenced by recent pollution spikes, delayed weather effects, or persistent pollution patterns.

The explainability outputs will be presented through visualizations such as station influence maps, attention-based graph plots, feature importance charts, and time-series plots. These explanations will make the model more transparent and useful for air quality monitoring, pollution interpretation, and decision support.

3.3 Flowchart

Figure 3-4 presents the overall workflow from data collection to dashboard visualization.

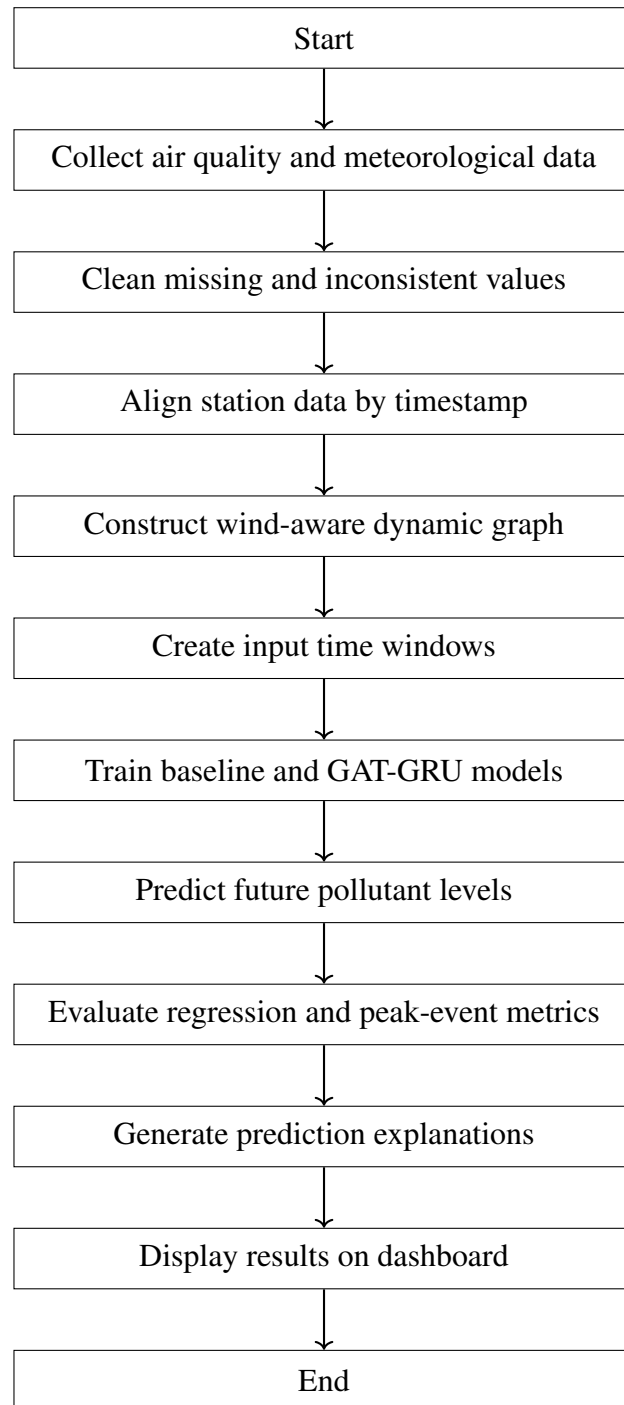


Figure 3-4 Proposed air quality forecasting workflow

3.4 Instrumentation Tools

Table 3-2 Proposed Technical Stack

Category	Tools and Technologies
Data Processing	Python, pandas, NumPy, scikit-learn
Deep Learning	PyTorch, PyTorch Geometric, SHAP
Forecasting Models	Persistence model, linear/tree regression, GRU, GAT-GRU
Data Sources	OpenAQ, Open-Meteo
Backend	FastAPI, Representational State Transfer (REST) services
Frontend	React, Leaflet or Mapbox, Recharts
Database and Storage	PostgreSQL, CSV/Parquet files, local storage for datasets, models, and experiment logs
Deployment	Docker, local server, optional cloud deployment

3.5 Evaluation Methodology

3.5.1 Evaluation Metrics

The project will evaluate forecasting models using regression metrics and horizon-wise error analysis. The proposed GAT-GRU model will be compared with baseline models to check whether dynamic graph learning improves forecasting accuracy.

Mean Absolute Error (MAE) measures the average absolute prediction error in $PM_{2.5}$ values:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Root Mean Squared Error (RMSE) penalizes large errors more strongly and is useful for evaluating pollution spikes:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Mean Absolute Percentage Error (MAPE) measures prediction error in percentage form:

$$MAPE = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100$$

The R^2 score shows how well the model explains variation in $PM_{2.5}$ concentration:

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

Bias indicates whether the model generally overpredicts or underpredicts:

$$Bias = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)$$

3.5.2 Peak Metrics

Peak-event metrics will be used to evaluate whether the system can identify high-pollution events. Precision measures how many predicted pollution events were actually correct:

$$Precision = \frac{TP}{TP + FP}$$

Recall, also known as probability of detection, measures how many actual pollution events were correctly detected:

$$Recall = \frac{TP}{TP + FN}$$

The F1 score balances precision and recall for event detection performance:

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

Explainability will be assessed qualitatively by inspecting whether important stations, weather factors, and time windows are consistent with wind direction, distance, and pollutant history. The dashboard will be evaluated based on whether it can clearly present forecasts, AQI categories, spatial maps, explanation visualizations, and what-if scenario comparisons.

4 EXPECTED OUTCOME

The project aims to deliver a working research prototype for short-term PM_{2.5} forecasting in selected Kathmandu Valley stations with explainable spatio-temporal modeling. The proposed system is expected to support air quality forecasting, interpretation, visualization, and decision-support analysis.

4.1 Technical Outcomes

The expected technical outcomes of the project are as follows:

- **Cleaned and Documented Dataset:** A cleaned and well-documented station dataset with station metadata, hourly pollutant observations, meteorological variables, preprocessing records, and model-ready input features.
- **Preprocessing Pipeline:** A structured preprocessing framework for timestamp synchronization, missing value handling, outlier detection, feature scaling, temporal window generation, and integration of OpenAQ and Open-Meteo data.
- **Wind-Aware Dynamic Graph Module:** A dynamic graph construction method that combines geographical distance, PM_{2.5} correlation, wind direction alignment, and wind speed to model changing station-to-station influence.
- **Virtual Node Estimation Mechanism:** A graph-based method for estimating PM_{2.5} concentration at unmonitored map locations using nearby real monitoring stations and wind-aware spatial relationships.
- **Baseline and Primary Forecasting Models:** Baseline models and a primary GAT-GRU forecasting model evaluated using standard regression metrics and peak-event detection metrics.
- **Explainability Module:** An explanation component that provides station influence analysis, SHAP-based feature importance, and temporal sensitivity analysis for interpreting model predictions.
- **Interactive Dashboard:** An interactive dashboard showing PM_{2.5} forecasts, AQI categories, spatial maps, station-wise trends, virtual-node estimates, and explanation visualizations.
- **Complete Project Report:** A complete project report including methodology, implementation details, performance analysis, limitations, feasibility discussion, and future enhancement possibilities.

5 PROJECT SCHEDULE

The project is planned for approximately one academic year. Work will begin with literature review and dataset selection, followed by data collection, preprocessing, graph construction, baseline models, Spatio-Temporal Graph Neural Network (ST-GNN) development, explainability, dashboard development, testing, documentation, and final presentation.

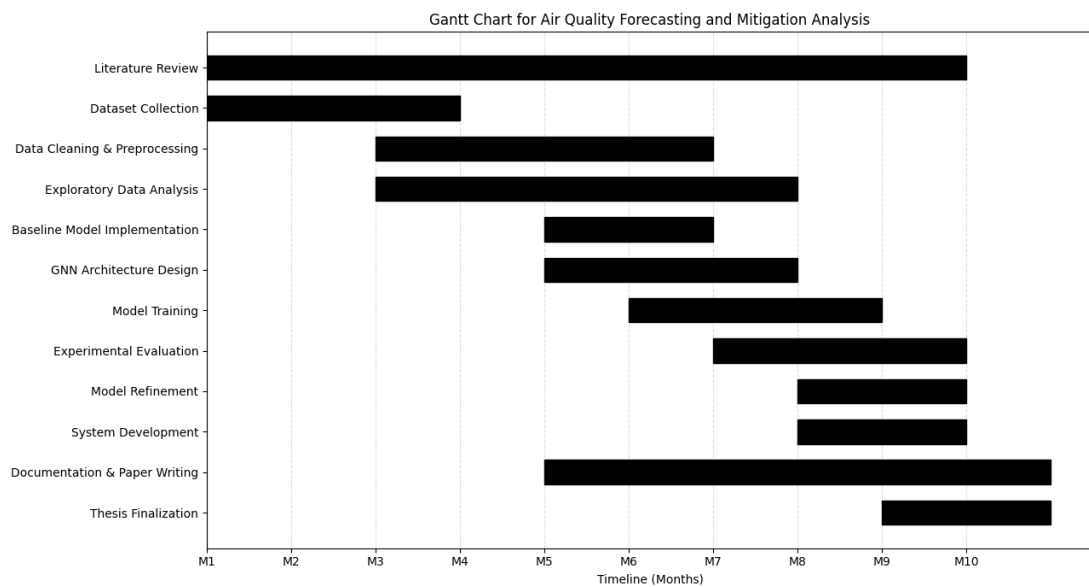


Figure 5-1 Gantt Chart

6 PROJECT BUDGET

The proposed system is mainly a software and research project, so the required budget is relatively low. Most libraries and datasets are open source or publicly available. The main expected expenses are internet access, optional cloud or Graphics Processing Unit (GPU) compute, deployment, documentation, and miscellaneous project needs.

Table 6-1 Estimated Project Budget

Category	Description	Estimated Cost
Internet and Data Access	Downloading datasets, accessing APIs, and reviewing research resources	NRs. 2,000
Cloud/GPU Usage	Optional Google Colab Pro, Kaggle, or equivalent compute for model training	NRs. 5,000
Domain/Hosting	Demo deployment for dashboard presentation, if needed	NRs. 3,000
Printing and Binding	Proposal, progress report, final report, and presentation materials	NRs. 6,000
Miscellaneous	Backup expenses and unforeseen project needs	NRs. 2,000
Total Estimated Cost		NRs. 18,000

Most software tools used in this project will be open source. GPU usage may be reduced by using free Google Colab or Kaggle Notebook resources during early experiments.

7 FEASIBILITY ANALYSIS

7.1 Technical Feasibility

The project is technically feasible because the required data sources, algorithms, and software tools are available. Air quality data can be collected from OpenAQ, while meteorological data such as wind speed, wind direction, temperature, humidity, pressure, and dew point can be obtained from Open-Meteo. These datasets can be integrated at an hourly resolution and used for PM_{2.5} forecasting in Kathmandu Valley.

The proposed model is also technically feasible because it uses established machine learning and deep learning tools. Data preprocessing can be performed using Python, pandas, NumPy, and scikit-learn. The forecasting models can be implemented using PyTorch and PyTorch Geometric. Baseline models such as persistence models, regression-based models, and GRU can be developed first, followed by the primary GAT-GRU model.

The use of a wind-aware dynamic graph provides sufficient technical depth for the project. The graph will represent monitoring stations as nodes and station relationships as edges. Edge weights will be constructed using geographical distance, PM_{2.5} correlation, wind direction, and wind speed. This allows the model to consider both spatial proximity and meteorological influence when forecasting PM_{2.5} concentration.

The virtual node component also adds research value while remaining technically manageable. It can be validated using a leave-one-station-out strategy, where one real monitoring station is temporarily treated as an unmonitored location and the predicted value is compared with its actual PM_{2.5} reading. This makes virtual-node evaluation possible even when ground truth is not available for every unmonitored location.

7.2 Operational Feasibility

The proposed system is operationally feasible as a research prototype. The system will operate as a web-based dashboard where users can view station-wise PM_{2.5} forecasts, AQI categories, spatial pollution maps, and explanation outputs. The backend service can provide processed data, model predictions, and explanation results through REST APIs, while the frontend can display the results through charts, maps, and station-level summaries.

The system does not require physical sensor deployment because it will initially use publicly available and downloadable air quality and meteorological data. This makes the project practical within an academic setting. The dashboard will be designed for interpretation, demonstration, and academic analysis rather than official public warning.

The operational workflow is also practical. Data can be collected and cleaned offline, models can be trained and evaluated using prepared datasets, and the trained model outputs can be displayed through the dashboard. This makes the system suitable for proposal defense, progress demonstration, and final project presentation.

7.3 Economic Feasibility

The project is economically feasible because most of the required tools and datasets are free or open-source. Python, pandas, NumPy, scikit-learn, PyTorch, PyTorch Geometric, FastAPI, React, and SHAP can be used without licensing cost. Public data sources such as OpenAQ and Open-Meteo can be accessed without major financial investment.

The main possible expenses are internet access, optional cloud or GPU usage, hosting for dashboard demonstration, printing, and documentation. The computational requirement is expected to be manageable because the model will be developed using selected Kathmandu Valley monitoring stations and hourly observations. Free platforms such as Google Colab, Kaggle Notebook, or local machines can be used during early experiments. The estimated budget is shown in Table 6-1.

7.4 Schedule Feasibility

The project is feasible within the academic project timeline if developed in phases. The first phase will focus on collecting and cleaning PM_{2.5} and meteorological data for selected Kathmandu Valley stations. The second phase will develop baseline models such as persistence, regression-based models, and GRU. The third phase will implement wind-aware graph construction and the GAT-GRU model. The fourth phase will add virtual-node estimation and explainability. The final phase will integrate the trained model with a dashboard and complete testing, documentation, and presentation.

The focused project scope improves schedule feasibility by allowing the team to concentrate on the most important technical components: data preparation, wind-aware graph construction, PM_{2.5} forecasting, virtual-node estimation, explainability, and dashboard visualization. This phased approach helps ensure that a working prototype can be completed within the academic project timeline while still maintaining sufficient research depth.

7.5 Risk and Mitigation

The major risks of the project are related to data quality, station availability, missing values, weather-data mismatch, model complexity, virtual-node validation, and dashboard integration. These risks and their mitigation strategies are summarized in Table 7-1.

Table 7-1 Project Risks and Mitigation Strategies

Risk	Mitigation Strategy
Missing or inconsistent PM _{2.5} readings	Select stations with sufficient data availability, apply timestamp alignment, use interpolation, and remove unreliable records.
Weather-data mismatch	Match each monitoring station with the nearest available weather grid point and align meteorological records by timestamp.
Limited station density	Start with selected stations that provide useful spatial coverage across Kathmandu Valley and validate graph behavior carefully.
Virtual-node validation difficulty	Use leave-one-station-out validation before applying virtual-node prediction to completely unmonitored locations.
Model complexity	Implement simple baseline models first, followed by GRU, wind-aware graph construction, and then the GAT-GRU model.
Long training time	Use selected Kathmandu Valley stations, hourly observations, efficient batching, and optional free GPU resources only when needed.
Dashboard integration complexity	Build a simple dashboard with station-wise forecasts first, then gradually add AQI categories, maps, explanation plots, and virtual-node outputs.

7.6 Limitations

The proposed system has some limitations. Prediction accuracy will depend on the quality, continuity, and availability of station data and meteorological data. Weather variables obtained from external sources may not perfectly represent local conditions at every monitoring station. Virtual-node predictions are estimates and may not fully replace real sensor measurements at unmonitored locations.

The model explanations will provide useful interpretation of station influence, feature importance, and temporal patterns, but they should be understood as model-based explanations rather than absolute physical causation. The system is intended as an academic research prototype for PM_{2.5} forecasting, explanation, and visualization. It will not replace official government air quality monitoring or warning systems.

7.7 Final Feasibility Statement

The proposed project is feasible as a major project because it has a focused and realistic scope while still maintaining strong technical depth. The use of wind-aware dynamic graph construction, GAT-GRU forecasting, virtual-node estimation, and explainability provides a meaningful research contribution for PM_{2.5} forecasting in Kathmandu Valley.

The required datasets, software tools, and development frameworks are available, and the project can be completed in phases within the academic timeline. Therefore, the proposed system is technically, operationally, economically, and schedule-wise feasible as a research prototype for explainable PM_{2.5} forecasting and visualization in Kathmandu Valley.

REFERENCES

- [1] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *International Conference on Learning Representations*, 2017.
- [2] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, “Graph attention networks,” in *International Conference on Learning Representations*, 2018.
- [3] S. Wang, Y. Li, J. Zhang, Q. Meng, L. Meng, and F. Gao, “PM2.5-GNN: A domain knowledge enhanced graph neural network for PM2.5 forecasting,” <https://arxiv.org/abs/2002.12898>, 2020.
- [4] M. Pandey, V. Jain, N. Godhani, S. N. Tripathi, and P. Rai, “Spatio-temporal forecasting of PM2.5 via spatial-diffusion guided encoder-decoder architecture,” <https://arxiv.org/abs/2412.13935>, 2024.
- [5] M. Panja, T. Chakraborty, A. Biswas, and S. Deb, “E-STGCN: Extreme spatiotemporal graph convolutional networks for air quality forecasting,” <https://arxiv.org/abs/2411.12258>, 2024.
- [6] L. Chen, J. Xu, B. Wu, M. Lv, C. Zhan, S. Chen, and J. Chang, “HighAir: A hierarchical graph neural network-based air quality forecasting method,” <https://arxiv.org/abs/2101.04264>, 2021.
- [7] J. Xu, S. Wang, N. Ying, X. Xiao, J. Zhang, Y. Cheng, Z. Jin, and G. Zhang, “Dynamic graph neural network with adaptive edge attributes for air quality predictions,” <https://arxiv.org/abs/2302.09977>, 2023.
- [8] K. H. Hettige, J. Ji, S. Xiang, C. Long, G. Cong, and J. Wang, “AirPhyNet: Harnessing physics-guided neural networks for air quality prediction,” in *International Conference on Learning Representations*, 2024.
- [9] R. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, “GNNExplainer: Generating explanations for graph neural networks,” in *Advances in Neural Information Processing Systems*, 2019, pp. 9240–9251.
- [10] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems*, 2017, pp. 4765–4774.
- [11] OpenAQ, “OpenAQ API Documentation: Measurements,” <https://docs.openaq.org/resources/measurements>, 2026, accessed: 2026-06-15.

- [12] Open-Meteo, “Historical weather api documentation,” <https://open-meteo.com/en/docs/historical-weather-api>, 2026, accessed: 2026-06-15.
- [13] S. Becker, R. P. Sapkota, B. Pokharel, L. Adhikari, R. P. Pokhrel, S. Khanal, and B. Giri, “Particulate matter variability in kathmandu based on in-situ measurements, remote sensing, and reanalysis data,” NOAA / repository report, Tech. Rep., 2020.
- [14] A. K. Panday and R. G. Prinn, “Diurnal cycle of air pollution in the kathmandu valley, nepal: Observations,” *Journal of Geophysical Research: Atmospheres*, vol. 114, p. D09305, 2009.
- [15] Clean Energy Nepal, “Policy brief on air quality and health in kathmandu valley,” Clean Energy Nepal / Urban Health Initiative, Kathmandu, Nepal, Tech. Rep., n.d.
- [16] ICIMOD, “Action plan to reduce air pollution and improve air quality,” International Centre for Integrated Mountain Development, Tech. Rep., 2021.